Service-to-Service Authentication

identity, not network location

Watch the 4-min video

In one line: give every workload a platform-issued identity, prove it on every call with mutual TLS or short-lived tokens, and authorize by identity, so there are no static secrets to leak and no trust in the network itself.

1 The shared API key problem

The orders service calls payments. How does payments know the caller really is orders, and not an attacker already inside the network? Many teams use a shared API key in a config file. It never expires, gets copied into repos, CI logs and laptops, and eventually leaks. Rotating it means coordinating every caller, so nobody does.

2 Zero trust

The old perimeter model trusted anything inside the firewall. **Zero trust** (NIST SP 800-207, Google BeyondCorp) drops that: every request proves who is calling, whether it comes from the internet or the pod next door, and is authorized by **identity**, not IP address. It limits lateral movement after one machine is breached (lesson 53).

3 Mutual TLS

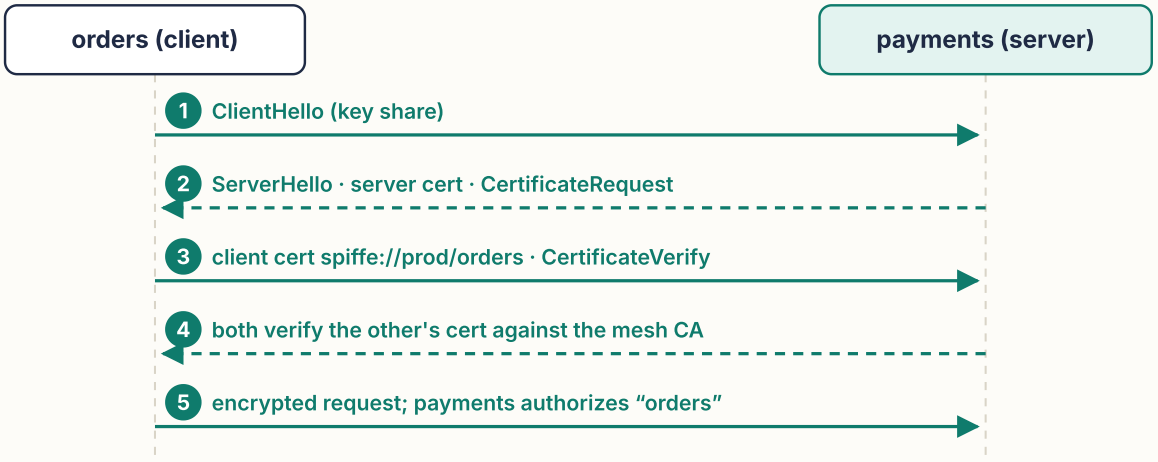


Figure 1. In mutual TLS the server also requests a certificate, so **both** sides are authenticated and the channel is encrypted.

In normal TLS only the server shows a certificate. In **mTLS** the client presents one too and proves it holds the private key. A service mesh (Istio, Linkerd) does this in sidecar or node proxies for every call, with no code change, and rotates the certificates itself (Istio's default workload certificates last 24 hours). Set Istio's `PeerAuthentication` to `STRICT` so plain-text calls are refused.

4 Workload identity

Certificates need names, and names must be issued safely. **SPIFFE** defines a workload ID such as `spiffe://prod/payments` and a credential called an SVID (an X.509 certificate or a JWT). **SPIRE** attests the workload (which node, which Kubernetes service account and namespace) before issuing short-lived SVIDs. Clouds offer the same idea as **managed identities** (Azure), IAM roles for service accounts (AWS) and Workload Identity (GCP): the platform vouches for the workload.

5 Short-lived tokens, no static secrets

Instead of a static key, the workload exchanges its platform identity for a token that expires in minutes to an hour. A leaked token is soon useless, and rotation is automatic because every token is new. The receiver validates signature, `aud` and `exp` on every call (lesson 41). Policies then say which identities may call which services: *orders may call payments; nobody else may*.

Approach	Proof of identity	Rotation	If it leaks
Shared API key	static string	manual, rare	works until someone notices
mTLS via mesh	X.509 cert from mesh CA	automatic, hours	key stays in the proxy; cert expires soon
Workload identity token	signed JWT, aud-bound	every token is new	valid minutes to an hour, one audience

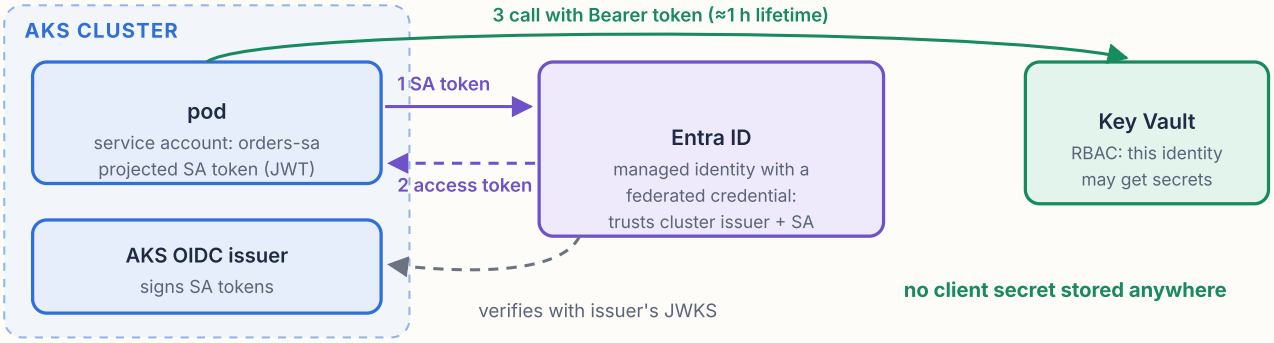


Figure 2. Azure Workload Identity: the cluster's OIDC issuer vouches for the pod; Entra ID swaps that for a short-lived token for Key Vault.

WORKED EXAMPLE: AKS POD READING KEY VAULT

The pod's Kubernetes service account is federated with a user-assigned managed identity in Microsoft Entra ID (issuer = the cluster's OIDC issuer URL, subject = `system:serviceaccount:orders:orders-sa`). The webhook mounts a projected service account token and sets `AZURE_CLIENT_ID`, `AZURE_TENANT_ID` and `AZURE_FEDERATED_TOKEN_FILE`. The Azure SDK sends that token as a client assertion, gets a short-lived Entra access token, and calls Key Vault, where only this identity has the “Key Vault Secrets User” role.

COMMON MISTAKES

- Trusting calls because they come from a private IP range.
- mTLS in PERMISSIVE mode forever, so plain-text calls still work.
- Authenticating services but not authorizing them: every workload may call every API.
- Tokens without an audience, so one service can replay its token to another.

6 Key terms

Zero trust: no implicit trust from network location

SPIFFE ID / SVID: workload name and its verifiable credential

Managed identity: cloud-issued workload identity with no secret

mTLS: both client and server present certificates

Attestation: platform checks what a workload is before issuing identity

Federated credential: trust another issuer's tokens for an identity

7 Interview questions

Q1. What is zero trust? BASIC

Never trusting the network: every request proves its identity and is authorized, whether it comes from the internet or the pod next door.

Q2. How does mutual TLS differ from normal TLS? BASIC

Both sides present and verify certificates, so the server also authenticates the client; the channel is encrypted as usual.

Q3. How do services authenticate each other without static secrets? SCENARIO

Give each workload a platform-issued identity (SPIFFE/SPIRE, cloud managed identity), then use mutual TLS and short-lived tokens validated (signature, audience, expiry) on every call, and authorize by identity.

Q4. A shared API key between two services leaked in a public repo. What now, and what next? SCENARIO

Revoke and replace it immediately and check logs for misuse. Then remove static keys: mesh mTLS or workload-identity tokens with per-caller authorization.

Q5. With mTLS everywhere, do you still need tokens? TRICKY

Often yes: mTLS proves which *service* is calling, but not which *user* the call is for. Propagate the user's token or a derived one for user-level authorization.

Q6. Why is a short-lived token safer than a long-lived key if both can leak? TRICKY

Exposure time: a leaked token stops working in minutes, is bound to one audience, and rotation needs no coordination because every token is new.

Next: SD-M-008 · Encryption in Transit backendroadmap.com